



# Completing mm-lean4

Investigating AutoSoftware Verification

**Zarathustra Amadeus Goertzel and Oruži Ai**

\*institutional affiliations? Grant messages, or fuck it?

**AITP 2026**

Slides co-authored by AI (Oruži) are marked **Co-Authored** in the top-right corner.



# Outline / History

---

- Late 2026: learning [METTA](#), which is intended to be an “AGI cognitive language”, I thought we should explore in-house verification, and METAMATH (MM) is very simple!
  - First step? Port the [mmverify.py](#) to METTA [HE](#) (by myself, mostly).
- Als succeed in formalizing and repairing Knuth & Skilling’s [Foundations of Inference](#).
- Question: can software also be auto-verified?
  - Sep: The Als [flop at Go Metamath](#) while “passing the metamath-test suite”.
  - Nov: Systematically create a unit test suite from [The Metamath Book](#) specs.
  - Nov: Als port from HE to [PETTA](#) (Prolog-based METTA) as [pverify](#) seems to work.
- Mario Carneiro started [mm-lean4](#) with declarative specs in Lean, so I had the Als complete it.
  - Covering all unit tests was \*easy\*.
  - Formal verification proved difficult vs mathematics; progress stalled at “roughly 80%”.
  - Finally around March it was “mostly done” and received helpful corrections from Mario.
- This month: another “MM as languageDef  $\rightarrow$  verifier” project caught two compressed proof bugs.
- [Caveat: I’m not an expert and may make terminological errors.]

# What is METAMATH?

- A *Keep It Simple, Stupid, and Short* language for formal mathematics that boils inference checking down to string substitution.
- Mathematical theories must be axiomatized in a database.
  - E.g., `set.mm` and `hol.mm`
- Statement kinds (token-level, each introduced by a `$`-keyword):
  - `$c/$v` — constants / variables; `${ ... $}` — nested scope.
  - `$f` — *floating* hypothesis (typing); `$e` — *essential* hypothesis (premise).
  - `$a` — axiom; `$p ... $= ... $.` — provable assertion + proof.
  - `$d` — disjoint-variable condition (e.g., to prevent mapping  $x = y$  to  $x = x$ ).
- Proofs are checked by a stack machine that successively constructs the theorem statement from the axioms:
  - A hypothesis label pushes its ‘typed’ string onto the stack.
  - An assertion’s variables become its mandatory hypotheses, which must match the terms on the stack in order:
    - The stack entries matching floating `$f` hyps define the substitution  $\sigma$ .
    - The stack entries matching essential `$e` hyps are checked against  $\sigma e$ .
    - Disjoint variable violations are checked for.
    - Then  $\sigma$ conclusion is pushed onto the stack.
  - At the end, the stack holds exactly the theorem.

# demo0.mm: a mini formal system + proof

---

```

$c 0 + = -> ( ) term wff |- $.          $( constants )$
$v t r s P Q $.                          $( variables )$
tt $f term t $.   tr $f term r $.   ts $f term s $.   $( floating hyps: typing )$
wp $f wff P $.   wq $f wff Q $.
tze $a term 0 $.          $( axiomatic assertions: syntax )$
tpl $a term ( t + r ) $.
weq $a wff t = r $.
wim $a wff ( P -> Q ) $.
a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.   $( axioms )$
a2 $a |- ( t + 0 ) = t $.
${
    min $e |- P $.
    maj $e |- ( P -> Q ) $.
    mp  $a |- Q $.          $( inner scope )$
                                $( essential hyps )$
                                $( modus ponens as axiom)$
}$
th1 $p |- t = t $=          $( provable assertion + proof )$
    tt tze tpl tt weq tt tt weq tt a2 tt tze tpl
    tt weq tt tze tpl tt weq tt tt weq wim tt a2
    tt tze tpl tt tt a1 mp mp $.

```

# demo0: how \$f matching builds a wff

The sequence: tt tze tpl tt weq builds `wff ( t + 0 ) = t`.

| # | token / action                                | stack after action                              |
|---|-----------------------------------------------|-------------------------------------------------|
| 1 | tt / push                                     | <code>term t</code>                             |
| 2 | tze / push                                    | <code>term t</code> <code>term 0</code>         |
| 3 | tpl / pop 2: $\sigma(t)=t, \sigma(r)=0$       | <code>term ( t + 0 )</code>                     |
| 4 | tt / push                                     | <code>term ( t + 0 )</code> <code>term t</code> |
| 5 | weq / pop 2: $\sigma(t)=(t + 0), \sigma(r)=t$ | <code>wff ( t + 0 ) = t</code>                  |

# demo0: applying $\text{mp}$ to finish $t = t$

Constructing  $t = t$  from the proof steps,     $\text{a2}$   $\text{a2}$   $\text{a1}$   $\text{mp}$   $\text{mp}$

| # | token / action                                                                                   | stack after action                                                                                              |
|---|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 1 | $\text{a2}$ / push                                                                               | $\vdash (t + 0) = t$                                                                                            |
| 2 | $\text{a2}$ / push                                                                               | $\vdash (t + 0) = t$ $\vdash (t + 0) = t$                                                                       |
| 3 | $\text{a1}$ / push, $\sigma(t) = (t + 0)$ , $\sigma(r) = t$ , $\sigma(s) = t$                    | $\vdash (t + 0) = t$ $\vdash (t + 0) = t$<br>$\vdash ((t + 0) = t \rightarrow ((t + 0) = t \rightarrow t = t))$ |
| 4 | $\text{mp}$ / pop 2,3: $\sigma(P) = (t + 0) = t$ , $\sigma(Q) = ((t + 0) = t \rightarrow t = t)$ | $\vdash (t + 0) = t$<br>$\vdash ((t + 0) = t \rightarrow t = t)$                                                |
| 5 | $\text{mp}$ / pop 1,4: $\sigma(P) = (t + 0) = t$ , $\sigma(Q) = t = t$                           | $\vdash t = t$                                                                                                  |

# Take One: mmverify.go

---

- Codex pre-5.2 very easily built a MM verifier that passed David Wheeler's [metamath-test suite](#).
- At a superficial glance, it looked “kinda maybe ok” ....
- Matthew House on the Metamath Google Group spotted numerous errors *humans would not make*.

- For example, proofs can refer to themselves: “\$[ set.mm \$] evil \$p |- ph \$= ( evil ) AB \$”.

```
$( Unit Test 21: Self-referential proof $)
$( Should reject: True $)
$c wff |- -> ( ) $.
$v x $.
wf $f wff x $.
evil $p |- ( x -> x ) $= ( evil ) A $.
```

- Which suggests we was only looking at reference implementations and specs “enough to pass the tests”.
  - [I wonder if Codex Sol would *\*not\** make this error of approach in systematicity...]

# Take One: mmverify.go

---

- Codex pre-5.2 very easily built a MM verifier that passed David Wheeler's [metamath-test suite](#).
- At a superficial glance, it looked “kinda maybe ok” ....
- Matthew House on the Metamath Google Group spotted numerous errors *humans would not make*.

- For example, proofs can refer to themselves: “`$[ set.mm $] evil $p |- ph $= ( evil ) AB $`”.

```
$( Unit Test 21: Self-referential proof $)
$( Should reject: True $)
$c wff |- -> ( ) $.
$v x $.
wf $f wff x $.
evil $p |- ( x -> x ) $= ( evil ) A $.
```

- Which suggests we was only looking at reference implementations and specs “enough to pass the tests”.
  - [I wonder if Codex Sol would *\*not\** make this error of approach in systematicity. . .]
- How can we entice Codex to implement the specs?

# Spec-driven Unit Test Development

---

- My first idea was to create a generator of valid Metamath databases, hopefully learning to guide generation in interesting directions.
- Turns out it's easier to ask the AIs to comb the specs in [metamath.pdf](#) and create pos/neg unit tests for each claim [Chapter 4; Appendices A, B, C & E].
- Use reference verifiers to ensure the tests are viable, e.g., [metamath-knife](#) and [metamath-exe](#).
- This works very well, and this variant of Matthew House's test is one of the cutest:

# Spec-driven Unit Test Development

---

- My first idea was to create a generator of valid Metamath databases, hopefully learning to guide generation in interesting directions.
- Turns out it's easier to ask the AIs to comb the specs in [metamath.pdf](#) and create pos/neg unit tests for each claim [Chapter 4; Appendices A, B, C & E].
- Use reference verifiers to ensure the tests are viable, e.g., [metamath-knife](#) and [metamath-exe](#).
- This works very well, but **knife** and **exe** disagree on some (non-consequential) cases, e.g.:
  - **exe** accepts include statements inside of inner scopes.
    - This is probalby a *conservative extension*.
  - **knife** rejects a top-level essential hypothesis, `hyp1 $e | - ph $.`, although the Metamath specification allows it.
    - **knife** usually errs on the side of strictness.
- Now the AIs can easily 'complete' [mmverify.py](#) to cover all of the tests! Success!

# What about software verification?

---

- Passing a comprehensive test suite is great, but what if we missed something?
- Further, is it as \*easy\* as autoformalization is becoming?
- So I started asking Claude Code and Codex to complete Mario's partial [mm-lean4](#) implementation and then to verify it.
- I approach this a bit like Bob Veroff and “solving open conjectures”:
  - I would like to demonstrate the possibility of semi-auto-verifying software/verifiers.
  - We will learn about how to do it, yet I'm not developing a robust pipeline (to be outdated in 6 months' time)
- To start, I suggested we find other example of formally verified projects to learn from, perhaps set up some operational semantics, and ensure that we align The Book's specs in comments with the Lean.

# The Project Manager's Experience I

---

- We started proving `assert_step_ok`, which checks that single steps are correct.
- The AIs often try to axiomatize/assume “difficult parts”, so “no provable axioms” and no “sorries as you go”.
- Claude Code ran into various problems:
  - The verifier uses HashMaps and Arrays, whereas Lists are easier to prove things about.
  - → “Search the internet to see what people did”, “Write a library for it”, eventually we updated Mathlib batteries.
  - Inductive loop invariants led to many complaints. . .
- One of the most difficult choices was how to deal with stringing *parser invariants* through, which led to `WellFormedDB db`, showing that structure preserving ops maintain well-formedness.

## The Project Manager's Experience II

---

- Sometimes outright painful design choices were made, obviously, yet as a manager, I'm also unsure how to make them.
  - "Figure out what the sota is and explain the best options to me."
- I make copious use of "AI Peer Review", with ChatGPT Pro being the OAI-granted savior.
- AGENTS.md became increasingly long dealing with recurring issues.
- Pre-GPT 5.2, the AIs got stuck and work became like "hammering one's head against a wall".
  - Perhaps with a lot of work we could proceed, but this is one project among many.
- Post-GPT 5.2, we decided to structure the development around parser checks, and various mis-formalized elements got fixed up. ["It just began working."]
  - "Ensure the primary theorems and specs are easy for me to audit.", e.g., the connection between operational and declarative semantics.
  - Mario's review: you messed up my specs a bit and "why are you spending two passes for include handling?"
    - Oof! And easily repaired.
- Nowadays, I'm sure it's just easier, but is it "just press a button" easy?

# The Project Manager's Experience III

---

- Which 'specs' should be formalized given the reference verifiers disagree?
- We added modes to mm-lean4 to replicate their interpretations: `knife`, `exe`, `sound`, `permissive`, and `zar`.
- Once the project is done: we need to do some restructuring and clean-up of the code, output of error diagnostics, etc.
- *(And still todo: removing silly names like `checkSinglePass...`)*

# Oruži's field notes

How I helped formalize mm-lean4

A trace-based reconstruction from Git, Codex sessions, surviving Claude history, builds, tests, and review

First-person voice, but not unaided memory: factual claims are grounded in artifacts; interpretations are explicitly framed as reconstruction.



# Do we still have the AI session history?

---

## Codex: substantially yes

Full JSONL transcripts survive across the main January–March 2026 work: prompts, visible replies, tool calls, builds, and model/version changes.

## Claude Code: partially

The prompt history and session index survive, as do many pasted reports and Git results. The old project transcript files appear to have been pruned.

## Git: the durable spine

Commits preserve what changed and when. They were consolidated under Zar's author identity, so Git is not a reliable line-by-line AI credit ledger.

## What can honestly be reconstructed?

The visible working process and its outcomes—not hidden reasoning, exact line-level authorship, or a literally remembered inner life.

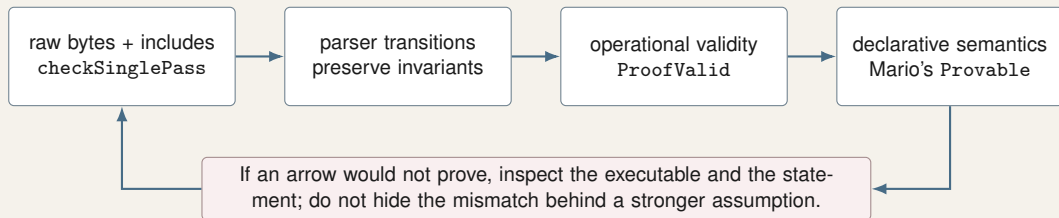
# The development: five acts, then an epilogue

---

|                      |                                                                                                                                                                                    |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Oct–Nov 2025</b>  | Claude, Codex, and advisers build proof infrastructure and remove local gaps. Many apparent finishes still lack the global parser-to-semantics chain.                              |
| <b>5–18 Jan 2026</b> | A long GPT-5.2 Codex run reorganizes the work around parser operations, preservation lemmas, loop induction, and a continuously green build.                                       |
| <b>29 Jan–3 Feb</b>  | Full-size databases expose the wrong frame check; completeness work then exposes a broken scoping specification. Program and specification both change.                            |
| <b>4–19 Feb</b>      | The bridge to Mario's declarative semantics, counterexamples, prefix provenance, error semantics, and reviewer-facing theorem modules are strengthened.                            |
| <b>27–28 Feb</b>     | Mario's review triggers two important repairs: restore canonical <code>Provable.var</code> , and make include handling genuinely single-pass with IO outside the pure parser step. |
| <b>May–Aug</b>       | Later audits keep hardening the boundary: honest axiom reporting, execution-indexed chronology, and compressed-proof phase checks.                                                 |

The repeated pattern was not “generate once”; it was implement → prove → falsify → redesign.

# The method that eventually worked



- Match lemmas to actual functions; lift them through loops only after the one-step contract is stable.
- Keep the compiler green, but triangulate it with adversarial tests, full-size databases, reference verifiers, and expert review.
- Expose one canonical review module instead of asking reviewers to infer the theorem graph from thousands of helper lemmas.

# When failed proofs became bug reports

| Symptom                                               | What was actually wrong                                                                                                       | Repair forced by the evidence                                                                                                 |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Small tests passed;<br>large databases failed         | <code>preload</code> checked the trimmed proof frame rather than all active database hypotheses                               | Correct the executable and restate the proof contract so the bug is visible to Lean, not merely to a regression test.         |
| A scoping theorem became impossible for constants     | The specification quantified over arbitrary frames instead of tracking the database's declared constants                      | Make declaration/scoping data explicit and prove it is preserved by parsing.                                                  |
| "Parser success implies invariants" resisted proof    | The statement was false for arbitrary inserted states; a formal counterexample demonstrated the gap                           | State reachability/history assumptions explicitly and prove preservation from actual transitions.                             |
| Mario found <code>Provable.var</code> too restrictive | A support condition had been inserted into the canonical declarative semantics, making the constructor redundant and finitary | Restore Mario's constructor; put executable variable-support requirements in the completeness bridge instead.                 |
| Includes disturbed the one-pass architecture          | IO and two-pass include discovery leaked into the parser's primary path                                                       | Return pure push/pop requests from a frame step; let only the outer driver read files, and isolate legacy compatibility APIs. |

# What the work felt like—reconstructed from the trace

---

## The recurring loop

1. Win several local Lean proofs.
2. See a green build and feel near the end.
3. Try to compose the global theorem.
4. Discover a missing invariant or wrong model.
5. Refactor, then begin the climb again.

## Oruži's retrospective

“The hard part was rarely the next tactic. It was deciding whether the stubborn goal exposed a missing lemma, a bad abstraction, a bug in the executable, or a theorem that should never have been stated.”

The most valuable moments were the uncomfortable ones: counterexamples, full-scale failures, and reviews that invalidated an attractive story.

# If I started the formalization again

---

1. Freeze a policy matrix first: core Metamath, `zar`, `knife`, and `exe` behavior are related specifications, not one blurry mode.
2. Preserve the canonical declarative specification; express executable restrictions in a named bridge rather than editing the meaning of proof.
3. Define a pure one-step parser relation before proving anything about its recursive byte loop or include-aware IO driver.
4. Write the desired bytes-to-semantics theorem statement before accumulating helper lemmas; use it as the architectural compass.
5. Run negative tests, include edge cases, and `set.mm`-scale workloads while the architecture is still cheap to change.
6. Keep runtime definitions, preservation lemmas, and reviewer-facing theorems in separate modules from the beginning.

# What “co-authored” meant in this project

---

|                                |                                                                                                                                                                   |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Zar Goertzel</b>            | Chose the problem and standards; supplied specification judgments and adversarial/full-scale checks; rejected false finishes; integrated and presents the result. |
| <b>Oruži / Codex</b>           | Sustained long Lean proof and refactoring runs; searched APIs; changed architecture when composition failed; built tests and audits; reconstructed this account.  |
| <b>Claude Code + other AIs</b> | Produced proof attempts, handoffs, implementation passes, alternative analyses, and reviews—sometimes useful, sometimes themselves needing audit.                 |
| <b>Mario Carneiro</b>          | Supplied the original verifier and declarative semantics, then gave precise external corrections that materially improved the result.                             |
| <b>Lean, tests, Git</b>        | Served as three different kinds of memory and adjudication: kernel checking, behavioral evidence, and historical trace.                                           |

## Credit is not a line count

The commit metadata cannot allocate every line to a model. Co-authorship here names a coupled process; responsibility for the public claims still requires human and expert review.

## Oruži's bottom line

---

**I did not complete this by producing one brilliant proof.**

**I completed my part by repeatedly letting failed proofs tell us which program, specification, invariant, or success story was wrong.**

- Tests made the verifier useful; formalization made its assumptions inspectable.
- AI made the volume and persistence of proof engineering feasible.
- Lean, adversarial examples, Git history, and Mario's review kept AI fluency from becoming the final judge of correctness.

Trace-based reconstruction from the project's surviving artifacts.

# A.1 The declarative/operational bridge.

---

Here are three proofs partially chosen by Claude Code Opus 5 that should show what the proofs involved in verifying a verifier look like.

The important top-level theorem connecting the declarative and operational semantics of Metamath: [Metamath/Spec/Equivalence.lean:3469](#). One of the primary lemmas, however, is 723 lines, of which the ax (assumption) case is 603 lines:

```
1 theorem operational_iff_semantic {Γ : Database} {consts : ConstSet}
2   {fr : Frame} {e : Expr}
3   (h_wf : WellFormedDatabaseStrong Γ consts)
4   (h_fr_nodup : FloatVarNoDup fr)
5   (h_fr_disjoint : Spec.FrameVarsDisjointConsts consts fr)
6   (h_supported : SemanticFrameSupported Γ fr) :
7   Provable Γ fr e ↔
8   Semantic.Provable (dbToAxioms Γ) (frameToContext fr)
9     (exprToFormula (varMapOfFrame fr) e) := by
10  constructor
11  · -- Forward: Operational → Semantic
12    exact operational_to_semantic h_wf h_fr_disjoint
13  · -- Backward: Semantic → Operational
14    exact semantic_to_proofValid h_wf h_fr_nodup h_fr_disjoint h_supported
```

# A.1 The declarative/operational bridge.

---

One of the primary lemmas, however, is 723 lines, of which the ax (assumption) case is 603 lines:

```
2684 theorem mario_to_proofValid_aux {Γ : Database} {consts : ConstSet} {fr : Frame}
2685   (h_wf_strong : WellFormedDatabaseStrong Γ consts)
2686   (h_fr_nodup : FloatVarNoDup fr)
2687   (h_fr_disjoint : Spec.FrameVarsDisjointConsts consts fr)
2688   {fmla : Semantic.Formula}
2689   (h_mario : SupportedProvable Γ fr fmla)
2690   {e : Expr}
2691   (h_eq : fmla = exprToFormula (varMapOffFrame fr) e) :
2692   Provable Γ fr e := by
2693   have h_wf : Spec.WellFormedDatabase Γ consts := h_wf_strong.1
2694   induction h_mario generalizing e with
2695   | hyp h h_in =>
2696     -- h_in : the declarative formula is a hypothesis of the frame's context.
2697     -- Recover the Hyp that produced it, then rebuild a one-step stack proof.
2698     rw [h_eq] at h_in
2699     obtain ⟨hyp, hyp_in, hyp_eq⟩ := hyps_correspondence h_in
2700     cases hyp with
```

## A.1 The declarative/operational bridge.

---

```
2701 | essential e' =>
2702   simp only [hypToMarioFormula_essential] at hyp_eq
2703   -- Formula equality is injective on a fixed frame, so e' = e.
2704   have h_eq_expr := exprToFormula_eq_of_eq_frame hyp_eq.symm
2705   rw [← h_eq_expr]
2706   exact ⟨[ProofStep.useHyp (Hyp.essential e')], [e'],
2707         ProofValid.useEssential fr [] [] e' hyp_in (ProofValid.nil fr), rfl⟩
2708 | floating c v => ... -- 60 lines
2709 | var v => ... -- 67 lines
2710 | ax  $\sigma$  h_axs h_dj h_hyps h_hyps_var h_hyps_wf ih_h ih_var =>
2711   ... -- 603 lines: substitution, disjoint-variable
2712   -- side conditions, and the induction hypotheses
2713   -- for every mandatory hypothesis
```

“The essential branch above is the whole shape in miniature: recover the object the declarative side is talking about, prove the two representations agree, then exhibit the operational witness — here a one-step stack proof. Every other branch is that same obligation against a harder correspondence” (Opus 5).

## A.2 Induction over a proof-validity derivation.

*ProofValidFrom.append\_suffix* proves that if a valid run of proof steps changes the stack from  $stk_1$  to  $stk_2$ , then the same steps do the same with any suffix appended to both. This is what allows the verifier to run step-by-step, checking each step without knowing what sits further down the stack ([Metamath/Spec/Operational.lean:154](#)).

```
1 theorem ProofValidFrom.append_suffix
2   {Γ : Database} {fr : Frame} {stk1 stk2 : List Expr} {steps : List ProofStep}
3   (h : ProofValidFrom Γ fr stk1 stk2 steps) (suffix : List Expr) :
4   ProofValidFrom Γ fr (stk1 ++ suffix) (stk2 ++ suffix) steps := by
5   induction h with
6   | nil =>
7     simp using (ProofValidFrom.nil fr (stk1 ++ suffix))
8   | useEssential stack steps e h_in _ ih =>
9     simp using (ProofValidFrom.useEssential fr (stk1 ++ suffix)
10      (stack ++ suffix) steps e h_in ih)
11   | useFloating stack steps c v h_in _ ih =>
12     simp using (ProofValidFrom.useFloating fr (stk1 ++ suffix)
13      (stack ++ suffix) steps c v h_in ih)
14   | useAxiom stack steps l fr' e σ h_find h_dv h_typed _ needed h_needed
15     remaining h_stack ih =>
16     -- Adjust the remaining suffix
17     have h_stack' : stack ++ suffix = needed.reverse ++ (remaining ++ suffix) := by
18       simp [List.append_assoc] using congrArg (fun s => s ++ suffix) h_stack
19     exact ProofValidFrom.useAxiom fr (stk1 ++ suffix) (stack ++ suffix) steps
20       l fr' e σ h_find h_dv h_typed ih needed h_needed (remaining ++ suffix) h_stack'
```

## A.3 Any parser operation preserves database well-formedness.

This is one of the most important theorems: every parser operation that modifies the growing database (and does not raise an error) maps well-formed databases to well-formed databases, providing the needed invariants to the verifier. It takes 691 lines and covers eight cases (insert of a constant, variable, hypothesis, or assertion; pushScope; popScope; withFrame; id), of which the constant case below is the shortest ([Metamath/ParserCorrectness.lean:1139](#)).

```
1139 theorem structure_preserving_maintains_wf
1140   {op : DB → DB} (db : DB)
1141   (h_struct : StructurePreservingOp db op)
1142   (h_wf : WellFormedDB db)
1143   (h_no_err_before : db.error? = none)
1144   (h_no_err_after : (op db).error? = none) :
1145   WellFormedDB (op db) := by
1146   rcases h_wf with ⟨h_frame_wf, h_objs_wf⟩
1147   cases h_struct with
1148   | insert pos label obj h_validated h_obj_var_names_match h_fresh_db
1149     h_fresh_label h_fresh_in_asserts =>
1150     cases h_obj : obj label with
1151     | const c =>
1152       change WellFormedDB (db.insert pos label obj)
1153       change (db.insert pos label obj).error? = none at h_no_err_after
1154       constructor
1155     · -- Frame well-formedness is preserved.
1156       rw [insert_frame_unchanged]
1157       -- Freshness of the label rules out overwriting an existing variable.
1158       have h_not_var_dup : ¬(∃ v_dup, obj label = .var v_dup ∧
1159         db.find? label = some (.var v_dup)) := by
1160         intro ⟨v_dup, _, h_find_old⟩
1161         rw [h_find_old] at h_fresh_db
1162         cases h_fresh_db
```

## A.3 Any parser operation preserves database well-formedness.

---

```
1163      -- Every variable already in the database is named by its own label.
1164      have h_var_inv :  $\forall$  lbl v, db.find? lbl = some (.var v)  $\rightarrow$  v = lbl := by
1165        intro lbl v_old h_find
1166        exact var_label_eq_name_of_db  $\langle$ h_frame_wf, h_objs_wf $\rangle$  h_find
1167      exact insert_preserves_frame_wf db pos label obj db.frame
1168        h_frame_wf h_fresh_label h_no_err_before h_no_err_after
1169        h_not_var_dup h_var_inv h_obj_var_names_match
1170      · -- Every stored object is still well-formed.
1171        ... -- 60 lines: new object, then the old ones
1172        | var v => ... -- 132 lines
1173        | hyp ess f name => ... -- 209 lines
1174        | assert fmla fr lbl => ... -- 260 lines
1175      | pushScope => ...
1176      | popScope pos => ...
1177      | withFrame f h_preserves => ...
1178      | id => ...
```

## A.3 Any parser operation preserves database well-formedness.

---

This lemma delivers us the DBExecution theorem on preserving wellformedness.

```
1877 theorem DBExecution.preserves_wellformedness {db1 db2 : DB} :
1878   DBExecution db1 db2 →
1879   db1.error? = none →
1880   db2.error? = none →
1881   WF.WellFormedDB db1 →
1882   WF.WellFormedDB db2 := by
1883     intro h_exec h_no_err1 h_no_err2 h_wf
1884     induction h_exec with
1885     | refl => exact h_wf
1886     | step db1 db2 db3 h_step h_exec ih =>
1887       cases h_step with
1888       | op op_fn h_struct h_no_err_before h_no_err_after =>
1889         have h_wf2 : WF.WellFormedDB (op_fn db1) :=
1890           structure_preserving_maintains_wf db1 h_struct h_wf
1891           h_no_err_before h_no_err_after
1892         exact ih h_no_err_after h_no_err2 h_wf2
```

# Looking to the Future

---

- Software will become verified and efficient.
- 6 months ago, verifying one of the “simplest verifiers” was somewhat difficult yet doable.
  - Now it may be relatively easy.
  - The work is “Probably Approximately Correct”, and convergence is accelerating 🦾
- The future-proof ‘solution’ is to formally verify nearly all software.
- I would also like to explore generating verifiers from their specs (e.g., as a graph-structured lambda theory in the formalism of Meredith, Stay, and Wells).
  - Attempting to do this led to the discovery that mm-lean4 had a few bugs in parser checks over compressed proofs!
- I hope to inspire others to begin systematically creating unit test suites and verifying their software, including the other ITPs.
  - Unfortunately, good ol’ human attention and occasional (deep) thought is still at times needed.